

LAB PRACTICE: EXCEPTIONS HANDLING AND THREADING

PART A: Exception Handling Fundamentals

Experiment A1: Basic Exception (try-catch)

To handle runtime exceptions using try-catch.

Program

```
class BasicException {
    public static void main(String[] args) {
        int a = 10, b = 0;

        try {
            int result = a / b;
            System.out.println(result);
        } catch (ArithmeticException e) {
            System.out.println("Error: Division by zero");
        }
    }
}
```

Task

- Change values of b
 - Observe when exception occurs
 - Identify type of exception
-

Experiment A2: Using throw (Manual Exception)

\\To generate exceptions explicitly using throw.

```
class ThrowExample {
    static void checkAge(int age) {
        if (age < 18) {
            throw new ArithmeticException("Not eligible");
        }
        System.out.println("Eligible");
    }

    public static void main(String[] args) {
        checkAge(16);
    }
}
```

```
}  
}
```

Task

- Modify condition
 - Create your own validation rule
-

Experiment A3: Using throws (Propagation)

Aim

To propagate exceptions using throws.

```
class ThrowsExample {  
  
    static int divide(int a, int b) throws ArithmeticException {  
        return a / b;  
    }  
  
    public static void main(String[] args) {  
        try {  
            int result = divide(10, 0);  
            System.out.println(result);  
        } catch (ArithmeticException e) {  
            System.out.println("Handled in main");  
        }  
    }  
}
```

Task

- Remove try-catch and observe behavior
 - Add multiple methods and propagate exception
-

PART B: Threading Fundamentals

Experiment B1: Basic Thread using Runnable

Aim

To create and execute a thread using Runnable.

```
class MyRunnable implements Runnable {  
  
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println("Thread: " + i);  
        }  
    }  
}  
  
public class ThreadBasic {  
    public static void main(String[] args) {  
        Thread t1 = new Thread(new MyRunnable());  
        t1.start();  
    }  
}
```

Task

- Replace start() with run() and observe difference
 - Add print statements in main
-

Experiment B2: Multiple Threads

Aim

To execute multiple threads concurrently.

```
class MyRunnable implements Runnable {  
    String name;  
  
    MyRunnable(String n) {  
        name = n;  
    }  
  
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println(name + ": " + i);  
        }  
    }  
}
```

```
public class MultiThreadDemo {
    public static void main(String[] args) {
        Thread t1 = new Thread(new MyRunnable("T1"));
        Thread t2 = new Thread(new MyRunnable("T2"));
        Thread t3 = new Thread(new MyRunnable("T3"));

        t1.start();
        t2.start();
        t3.start();
    }
}
```

Task

- Observe output order
 - Explain why output is not sequential
-

PART C: Exception Handling in Multithreading

Experiment C1: Exception inside Thread (No Handling)

Aim

To observe behavior of unhandled exceptions in threads.

```
class Task implements Runnable {

    int num, denom;

    Task(int n, int d) {
        num = n;
        denom = d;
    }

    public void run() {
        int result = num / denom;
        System.out.println("Result: " + result);
    }
}
```

Task

- Create threads with denominator = 0
 - Observe which thread fails
-

Experiment C2: Using try-catch inside Thread

```
public void run() {
    try {
        int result = num / denom;
        System.out.println("Result: " + result);
    } catch (ArithmeticException e) {
        System.out.println("Exception handled in thread");
    }
}
```

Note

- Exception must be handled **inside thread**
-

Experiment C3: Using throw and throws in Threads

```
class Task implements Runnable {

    int num, denom;

    Task(int n, int d) {
        num = n;
        denom = d;
    }

    int divide() throws ArithmeticException {
        if (denom == 0) {
            throw new ArithmeticException("Invalid input");
        }
        return num / denom;
    }

    public void run() {
        try {
            int result = divide();
            System.out.println(Thread.currentThread().getName() + ": " + result);
        } catch (ArithmeticException e) {
            System.out.println(Thread.currentThread().getName() + ": " + e.getMessage());
        }
    }
}
```

Main Class

```
public class ThreadExceptionFinal {
    public static void main(String[] args) {

        Thread t1 = new Thread(new Task(10, 2), "T1");
```

```
Thread t2 = new Thread(new Task(10, 0), "T2");
Thread t3 = new Thread(new Task(20, 5), "T3");

t1.start();
t2.start();
t3.start();
}
}
```

You should Observe

1. Threads execute independently
 2. Exception in one thread does not stop others
 3. Exception handling must be done within thread
 4. throws passes responsibility within thread context
-

Viva Questions

1. Why do exceptions not propagate to the main thread?
2. Difference between throw and throws in threading?
3. What happens if try-catch is removed from run()?
4. Why is start() preferred over run()?
5. Can multiple threads share the same exception handling logic?